Peppr Around — Genesis

> *“This folder is the soul of the platform. It contains everything we learned, everything we built, and everything we would do differently. Read it before you write a single line of code.”*

What Is This?

The Genesis folder is the living documentation of the Peppr Around platform — a hospitality service request management system built from scratch over 78 phases between January and March 2026. It was created as a Hall of Fame retrospective: a complete record of the platform’s DNA so that any future incarnation starts with wisdom rather than from zero.

This is not a technical specification. It is a **memory** — the accumulated knowledge of every decision made, every mistake corrected, and every principle discovered during the building of this platform.

Documents

File	Purpose
`00-MANIFESTO.md`	The soul — why this platform exists, what it believes, and what it will never compromise on
`01-ARCHITECTURE.md`	The skeleton — the system design, data model, and key technical decisions with rationale
`02-LESSONS-LEARNED.md`	The scars — an honest account of every significant mistake and discovery, categorised by domain
`03-PHASE-CHRONICLE.md`	The memory — a complete timeline of every major feature, pivot, and milestone across 78 phases
`04-DESIGN-LANGUAGE.md`	The aesthetic — the visual identity, component patterns, interaction principles, and copy rules
`05-REBIRTH-PLAYBOOK.md`	The blueprint — a 10-sprint guide to rebuilding the platform from scratch using this DNA

How to Use This Folder

If you are starting a new version of this platform: Read the documents in order. Start with the Manifesto to understand the purpose. Read the Architecture to understand the structure. Read the Lessons Learned to avoid the mistakes. Read the Phase Chronicle to understand the evolution. Read the Design Language to understand the aesthetics. Then follow the Rebirth Playbook sprint by sprint.

If you are joining the existing platform: Read the Manifesto and the Lessons Learned first. They will give you the context to understand why the code looks the way it does and what constraints you must respect.

If you are evaluating this platform as a product: Read the Manifesto and the Phase Chronicle. They tell the story of what was built and why.

Project Statistics at Phase 78

Metric	Value
Total phases	78 (+ sub-phases)
Git commits	231
Lines of TypeScript/TSX	84,000
Database tables	32
tRPC procedures	85
Unit tests	233
Pages (admin + guest)	45
Supported guest languages	8
OWASP compliance	Full Top 10 addressed
CI pipeline	Green

Credits

Product Director: Chawakit Sangcharoon
Engineering Partner: Manus AI
Period: January – March 2026
Repository: CiaoKit1444/Around-M-v01
Production URL: bo.peppr.vip

“The best time to plant a tree was 20 years ago. The second best time is now.”

Peppr Around — Genesis Manifesto

> *“We did not build a hotel management system. We built a language — a shared vocabulary between guests who need things and staff who deliver them.”*

What This Platform Is

Peppr Around is a QR-first, multi-property hospitality service platform. A guest scans a code on their door, browses a curated service menu, submits a request in their own language, and watches it move through a live status pipeline. On the other side of that pipeline, front office staff, service providers, and service operators each see only what they need to act on — no noise, no ambiguity, no missed requests.

The platform is not a booking engine. It is not a PMS. It is the **last mile of hospitality** — the moment a guest has a need and a human being responds to it. Every design decision, every data model, every copy string was made in service of making that moment faster, clearer, and more human.

The Problem We Solved

Traditional hotel service requests travel through a telephone, a walkie-talkie, or a paper log. Each hop loses information. By the time a request reaches the person who can fulfil it, the guest’s original words have been paraphrased, the urgency has been diluted, and accountability has evaporated.

Peppr Around eliminates those hops. The guest’s exact words, the timestamp of their request, the room they are in, and the service they need travel as a structured record — immutable, traceable, and actionable — from the moment of submission to the moment of completion.

The Five Principles That Must Never Be Compromised

1. The Guest Is Never Lost

Every guest interaction begins with a QR code scan. That scan creates a **session** — a lightweight, anonymous identity that persists for the duration of the stay. The guest never logs in. They never create an account. They simply scan and act. If the session expires, the QR code creates a new one. The guest is never asked to do anything technical.

2. The Hierarchy Is Sacred

The data model has exactly three levels: **Partner → Property → Room**. A Partner is a hotel management company. A Property is a hotel. A Room is a service unit. This hierarchy is not a convenience — it is the load-bearing structure of every permission, every filter, every report, and every notification in the system. Any future feature that violates this hierarchy will create debt that compounds.

3. Roles Are Portals, Not Permissions

The platform has five operational roles: Super Admin, Admin, Front Office, Service Provider, and Service Operator. Each role is a **portal** — a distinct view of the world with its own navigation, its own data scope, and its own vocabulary. A Front Office officer does not see the service catalog. A Service Provider does not see the front office queue. This is not a limitation; it is a feature. Cognitive load is the enemy of fast response.

4. Every Status Must Tell a Story

A service request moves through a defined lifecycle: Pending → Confirmed → In Progress → Completed (or Rejected / Cancelled). Each transition is a story beat. The guest tracking page must always answer the question: *"What is happening right now, and what happens next?"* A status label that only names the current state without describing the next action is a failed status label.

5. The Platform Must Be Operable Without Training

A new front office staff member should be able to handle their first request within five minutes of opening the dashboard for the first time. This is the operability test. If any workflow requires a manual, the workflow has failed.

The DNA of the Platform

The following properties define the platform's identity and must be preserved in any future incarnation.

Property	Value
Primary interaction model	QR scan → session → service request → status tracking
Identity model	Multi-tenant (Partner → Property → Room hierarchy)
Authentication	Manus OAuth (SSO) for staff; anonymous session tokens for guests
Data transport	tRPC (type-safe, end-to-end, no REST boilerplate)
Real-time	Server-Sent Events (SSE) for live request updates and presence
Design system	shadcn/ui + Tailwind CSS 4 + Geist font
Primary colour	Peppr Purple (`#7C3AED` / OKLCH equivalent)
Guest language support	8 languages (EN, TH, ZH, JA, KO, FR, DE, RU)
Deployment target	Manus hosted platform (`bo.peppr.vip`)
Test philosophy	233+ unit tests, CI-gated on every push

What We Learned About the Domain

Hospitality operations are **interrupt-driven**. Staff do not sit at a desk waiting for requests. They are moving through a building, handling multiple simultaneous tasks,

and switching context constantly. The platform must therefore:

- Surface the most urgent item at the top, always.
- Make the primary action (Confirm / Start / Complete) reachable in one tap.
- Never require a staff member to navigate more than two levels deep to act on a request.
- Provide ambient awareness (notification badges, sound alerts, pending request banners) so staff know something needs attention without actively checking.

These are not nice-to-haves. They are the difference between a platform that gets used and one that gets abandoned.

A Note to Future Builders

If you are reading this document as part of a rebuild, know that the platform you are inheriting was built in **78 phases** over a sustained period of iterative development. It started as a static mockup with hardcoded data and evolved into a production system with a live database, real-time SSE, a full CI pipeline, 2FA authentication, OWASP-compliant security headers, and a guest microsite supporting 8 languages.

The mistakes are documented in `02-LESSONS-LEARNED.md`. The architecture is in `01-ARCHITECTURE.md`. The full phase chronicle is in `03-PHASE-CHRONICLE.md`. The design language is in `04-DESIGN-LANGUAGE.md`. The rebuild playbook is in `05-REBIRTH-PLAYBOOK.md`.

Do not start from scratch. Start from here.

Document authored by Manus AI in collaboration with Chawakit Sangcharoon (Product Director), March 2026.

Peppr Around — Architecture Reference

> *This document describes the system as it exists at Phase 78 (March 2026). It is both a reference and a justification — every major decision is accompanied by the reasoning that produced it.*

System Overview

Peppr Around is a **monorepo full-stack web application** deployed on the Manus hosting platform. The system has two distinct user-facing surfaces: an **Admin/Staff Back-Office** (`bo.peppr.vip`) and a **Guest Microsite** (accessed via QR code deep-link). Both surfaces are served by the same Express server, share the same database, and communicate via the same tRPC API layer.

```
┌──────────────────────────────────────────────────────────────┐
│                    bo.peppr.vip                              │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  React 19 SPA (Vite)                                 │    │
│  │  ├── /admin/*   Admin Back-Office                    │    │
│  │  ├── /fo/*      Front Office Portal                  │    │
│  │  ├── /sp/*      Service Provider Portal              │    │
│  │  ├── /so/*      Service Operator Portal              │    │
│  │  └── /guest/*   Guest Microsite (QR deep-link)       │    │
│  └──────────────────────────────────────────────────────┘    │
│                          │ tRPC + SSE                         │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Express 4 Server                                    │    │
│  │  ├── /api/trpc        tRPC router                    │    │
│  │  ├── /api/sse         Server-Sent Events             │    │
│  │  ├── /api/oauth/*     Manus OAuth callback           │    │
│  │  └── /api/stripe/*    Stripe webhooks                │    │
│  └──────────────────────────────────────────────────────┘    │
│                          │ Drizzle ORM                        │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  MySQL / TiDB (managed)                              │    │
│  └──────────────────────────────────────────────────────┘    │
│                          │                                    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Redis (managed)  — rate limiting, JWT revocation,   │    │
│  │                     2FA tokens, session blacklist     |    |
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

The Data Hierarchy

The three-level hierarchy is the structural backbone of the entire system. Every piece of data is anchored to one of these three levels.

```
Partner (Hotel Management Company)
  └── Property (Hotel / Resort)
        └── Room / Service Unit (Guest-facing QR endpoint)
```

This hierarchy drives:

- **Authentication scope** — a staff member's `propertyId` determines which requests they see.
- **QR code generation** — every QR code is bound to a specific Room within a specific Property.
- **Notification routing** — inbox notifications are scoped to a property and filterable by partner.
- **Reporting** — all analytics aggregate up through the hierarchy.

Core Entity Map

Entity	Table	Key Relationships
Partner	`peppr_partners`	Has many Properties
Property	`peppr_properties`	Belongs to Partner; has many Rooms, Config, Banners
Property Config	`peppr_property_config`	1:1 with Property; guest branding, feature flags
Room / Service Unit	`peppr_rooms`	Belongs to Property; has one active Template Assignment
Service Provider	`peppr_service_providers`	Belongs to Property; handles SP tickets
Catalog Item	`peppr_catalog_items`	Belongs to Property; the menu of services
Service Template	`peppr_service_templates`	Belongs to Property; a named bundle of Catalog Items
Template Item	`peppr_template_items`	Join table: Template ↔ Catalog Item
Room Template Assignment	`peppr_room_template_assignments`	Join: Room ↔ Template (one active per room)
QR Code	`peppr_qr_codes`	Belongs to Room; has access type, expiry, rotation
Stay Token	`peppr_stay_tokens`	Belongs to QR Code; short-lived guest session key
Guest Session	`peppr_guest_sessions`	Created on QR scan; holds locale, font size pref
Service Request	`peppr_service_requests`	Belongs to Property + Room + Guest Session
Request Item	`peppr_request_items`	Line items within a Service Request

Entity	Table	Key Relationships
Request Event	`peppr_request_events`	Immutable audit trail of status transitions
Request Note	`peppr_request_notes`	Staff notes thread on a request
Staff Member	`peppr_staff_members`	Belongs to Property; has Position
SP Assignment	`peppr_sp_assignments`	Assigns a request to a Service Provider
SP Ticket	`peppr_sp_tickets`	Work ticket for a Service Provider
SO Job	`peppr_so_jobs`	Job for a Service Operator
Payment	`peppr_payments`	Stripe-linked payment for a request
Inbox State	`peppr_inbox_state`	Per-user read/mute state for notifications
Archived Notification	`peppr_archived_notifications`	Archived inbox items

Technology Stack Decisions

Why tRPC Instead of REST

The project began with a FastAPI (Python) backend and a REST API layer. This created a persistent friction: every time a backend endpoint changed, the frontend TypeScript types had to be manually updated. Type drift caused silent runtime errors that were difficult to trace.

The migration to tRPC (Phase 70 — “Pure JS/TS Stack Cleanup”) eliminated this entirely. Procedures defined in `server/routers.ts` produce types that are consumed directly by `trpc.*.useQuery/useMutation` hooks in the frontend. There is no contract

file, no code generation step, and no possibility of type mismatch. The compiler enforces the contract at build time.

Rule for future builders: Never introduce a REST endpoint for a new feature. Define a tRPC procedure. If you need a webhook endpoint (e.g., Stripe), register it as a raw Express route before the tRPC middleware.

Why MySQL / TiDB Instead of SQLite

The initial template used SQLite. SQLite is excellent for local development but has no support for concurrent writes in a production multi-tenant environment. MySQL/TiDB was chosen because it is the managed database provided by the Manus platform, supports the full Drizzle ORM feature set, and handles concurrent connections from multiple staff sessions without locking.

Schema management: All schema changes go through `pnpm db:push` which runs `drizzle-kit generate && drizzle-kit migrate`. Never alter the database directly. The migration files in `drizzle/migrations/` are the source of truth.

Why Server-Sent Events Instead of WebSockets

Real-time updates (new requests appearing in the front office queue, presence indicators on the request detail page) were initially considered for WebSocket implementation. SSE was chosen instead for three reasons:

1. SSE is unidirectional (server → client), which matches the actual data flow pattern — the server pushes state changes, the client does not push raw events back.
2. SSE works over standard HTTP/1.1 without protocol upgrade, making it compatible with the Manus proxy layer.
3. SSE connections are automatically reconnected by the browser on drop, with no client-side reconnection logic required.

The SSE endpoint at `/api/sse` broadcasts events scoped to `propertyId`, ensuring that staff at Hotel A do not receive events from Hotel B.

Why Manus OAuth Instead of Custom Auth

The platform originally used a custom FastAPI JWT authentication system with email/password login, 2FA (TOTP + SMS), and a full password reset flow. This was migrated to Manus OAuth in Phase 70 for the primary staff authentication path.

The custom auth system (`pepprUsers`, `pepprUserRoles`) was retained in the database because it handles the **Peppr-specific role model** — the multi-role binding system where a single user can be a Partner Admin for one hotel group and a Front Office officer for a specific property. Manus OAuth handles the SSO identity; the `pepprUsers` table handles the role-to-property binding.

The bridge: On first OAuth login, the system looks up the Manus `openId` in `pepprUsers.manusOpenId`. If found, the full Peppr profile (roles, propertyId, partnerId) is hydrated. If not found, a new `pepprUser` record is created with a default `USER` role.

Why Redis

Redis was added in Phase 46 to support:

- **JWT revocation** — a blacklist of invalidated tokens, checked on every protected request.
- **Rate limiting** — per-IP and per-user rate limits on authentication endpoints, enforced via Redis counters with TTL.
- **2FA token storage** — TOTP backup codes and SMS OTP tokens stored with short TTL.
- **Session management** — active session tracking for the “force logout all devices” feature.

Redis is not used as a primary data store. All business data lives in MySQL. Redis is the **ephemeral security layer**.

Authentication Flow

```
Staff Login (Manus OAuth):
  1. Frontend calls getLoginUrl() → redirects to Manus OAuth portal
  2. Manus OAuth redirects to /api/oauth/callback with code + state
  3. Server exchanges code for Manus user profile
  4. Server looks up pepprUsers by manusOpenId → hydrates Peppr roles
  5. Server sets HttpOnly cookie (JWT signed with JWT_SECRET)
  6. Frontend reads auth state via trpc.auth.me.useQuery()

Guest Session (QR Scan):
  1. Guest scans QR code → navigates to /guest/scan/:qrId
  2. Server validates QR code (active, not expired, not revoked)
  3. Server creates or resumes peppr_guest_sessions record
  4. Session token stored in localStorage (not a cookie)
  5. All guest API calls include session token in header
```

URL Structure

The platform uses a single domain with path-based portal separation. This was a deliberate decision made in Phase 21 (Server & Resource Normalization) to avoid subdomain complexity.

Path prefix	Portal	Authentication
`/admin/*`	Admin Back-Office	Manus OAuth + Admin role
`/fo/*`	Front Office Portal	Manus OAuth + FO role
`/sp/*`	Service Provider Portal	Manus OAuth + SP role
`/so/*`	Service Operator Portal	Manus OAuth + SO role
`/guest/*`	Guest Microsite	Anonymous (session token)
`/api/trpc`	tRPC API	Cookie (staff) or session token (guest)
`/api/sse`	Server-Sent Events	Cookie (staff only)

File Structure Conventions

```
server/
  routers.ts          ← tRPC procedures (split into sub-files when > 150
lines)
  crudRouter.ts       ← CRUD procedures for all hierarchy entities
  db.ts               ← Drizzle query helpers (reused across procedures)
  storage.ts          ← S3 helpers (storagePut, storageGet)
  _core/              ← Framework plumbing — DO NOT edit unless extending
infra

client/src/
  pages/              ← Page-level components (one file per route)
  components/         ← Reusable UI components
  components/shared/  ← Cross-page shared components (PageHeader,
EmptyState, etc.)
  components/ui/      ← shadcn/ui primitives
  components/dialogs/ ← Modal dialogs (BulkRoomCreate, QRBatchGenerate,
etc.)
  components/guest/   ← Guest microsite specific components
  contexts/           ← React contexts (AuthContext, NotificationContext)
  hooks/              ← Custom hooks (useActiveProperty, useAuth, etc.)
  lib/                ← Utilities (trpc.ts, getRoleLandingPath.ts)

drizzle/
  schema.ts           ← Single source of truth for all table definitions
  migrations/         ← Auto-generated migration files (never edit manually)

Genesis/              ← This folder — project DNA and retrospective
docs/                 ← Technical documentation (routes.md, schema.md, ADRs)
```

Security Architecture

The platform underwent a full OWASP audit in Phases 42–45. The following controls are in place:

Control	Implementation
HTTPS enforcement	Manus platform handles TLS termination
Security headers	Helmet.js (CSP, HSTS, X-Frame-Options, etc.)
CORS	Allowlist-based, configured via `CORS_ALLOWED_ORIGINS` env
Rate limiting	Redis-backed per-IP limits on `/api/trpc/auth.*`
JWT revocation	Redis blacklist checked on every protected procedure
2FA	TOTP (authenticator app) + SMS OTP; enforced per-user
Password policy	Min 12 chars, complexity rules, bcrypt hashing
Body size limit	10MB limit on all POST endpoints
SQL injection	Drizzle ORM parameterised queries — no raw SQL strings
XSS	React's default escaping + CSP headers
Audit log	Every state-changing action recorded in `peppr_audit_events`

Document authored by Manus AI in collaboration with Chawakit Sangcharoon (Product Director), March 2026.

Peppr Around — Lessons Learned

> *"Every mistake we made was a lesson we earned. This document exists so the next version of this platform does not have to earn the same lessons twice."*

How to Read This Document

This is an honest retrospective. Each lesson is categorised as either a **mistake** (something we did wrong and corrected), a **discovery** (something we learned that was

not obvious at the start), or a **principle** (a rule that emerged from repeated experience). The lessons are grouped by domain.

Architecture & Technology

Mistake: Starting with a Python/FastAPI Backend

The platform was originally built with a FastAPI backend (Python) serving a React frontend. This created a persistent type-safety gap: the frontend TypeScript types had to be manually maintained to match the Python Pydantic models. Every API change required updating two separate type systems. Silent type drift caused runtime errors that were hard to trace.

What we did: Migrated to a pure TypeScript stack (tRPC + Express) in Phase 70. The migration took significant effort but eliminated the type gap permanently.

Lesson: For a product built by a small team where the same person touches both frontend and backend, a shared-type RPC layer (tRPC, GraphQL with codegen) is not a luxury — it is a prerequisite for sustainable velocity. The cost of the migration was higher than the cost of starting with tRPC would have been.

Mistake: Hardcoded Property IDs in Early Phases

In the early phases (1–23), many components used a hardcoded `propertyId` of `"pr-001"`. This was a shortcut taken during rapid prototyping that became a critical bug when the platform was connected to real data. A full audit was required in Phase 24 to find and replace every hardcoded reference.

What we did: Built a `useActiveProperty()` hook that reads the property from the authenticated user context, with a fallback to the first available property for super-admins.

Lesson: Never hardcode entity IDs, even in prototype phases. The cost of replacing them later is always higher than the cost of wiring them correctly from the start. If the real value is not available yet, use a clearly-named constant (`DEFAULT_PROPERTY_ID`) that can be found and replaced with a single grep.

Mistake: Storing Demo Data in Component State

In phases 1–25, many pages rendered demo data from hardcoded arrays inside the component file. This created a false sense of completeness — the UI looked finished but was not connected to any real data source. When real data was wired up, the UI often broke because the demo data had different shapes than the real API responses.

What we did: Replaced all demo data with real tRPC queries, adding loading and empty states.

Lesson: Demo data in component state is technical debt that compounds. A page with a loading skeleton and an empty state is more honest and more useful than a page with hardcoded fake data. Build the real data layer first, even if it means the UI is empty for a while.

Discovery: SSE Is Better Than WebSockets for This Use Case

The real-time requirements of the platform (new requests appearing in the queue, presence indicators on detail pages) were initially assumed to require WebSockets. After evaluating the actual data flow, SSE was chosen instead.

The key insight was that the data flow is **asymmetric**: the server pushes state changes to the client, but the client does not push raw events back to the server. Clients mutate state through tRPC mutations (HTTP), not through the real-time channel. SSE is a perfect fit for this pattern — it is simpler, requires no protocol upgrade, and reconnects automatically.

Lesson: Before choosing WebSockets, ask whether the real-time channel is truly bidirectional. If the client only needs to receive server-pushed updates, SSE is simpler and more reliable.

Mistake: Not Setting Up CI from the Beginning

The GitHub CI pipeline (unit tests + integration tests) was set up in Phase 28. Before that, there was no automated gate on code quality. Several regressions were introduced and caught only by manual testing.

What we did: Added a `vitest` test suite and a GitHub Actions workflow that runs on every push to `main`.

Lesson: CI should be the first thing set up after the project scaffold, not the last. The cost of writing tests is always lower when the code is fresh. Tests written after the fact are always harder to write and less complete.

Mistake: GitHub Repository Secrets Not Configured

The CI pipeline failed on commit `b8aaff5` because `JWT_SECRET` and `SSO_BRIDGE_SECRET` were not set as GitHub repository secrets. The tests passed locally but failed in CI because the environment was not configured.

Lesson: When setting up CI, immediately document all required environment variables and add them to the CI environment. Create a `.env.example` file that lists every required variable (without values) so that any new environment (CI, staging, production) can be configured from a checklist.

Data Modelling

Principle: The Hierarchy Is the Schema

The three-level hierarchy (Partner → Property → Room) is not just a UI concept — it is the load-bearing structure of the entire data model. Every table that stores business data has a `property_id` or `partner_id` foreign key. This was not planned from the start; it emerged from repeated experience of needing to scope data to a property.

Lesson: Identify the primary tenancy boundary early and encode it in every table from the beginning. Adding `property_id` to a table that was created without it is a migration that touches every query and every index.

Mistake: Not Separating Request Events from Request State

In early phases, the status of a service request was a single `status` field on the `peppr_service_requests` table. This was simple but made it impossible to answer

questions like “how long did this request spend in the Pending state?” or “who confirmed this request and when?”

What we did: Added `peppr_request_events` — an immutable append-only table that records every status transition with actor, timestamp, and reason. The `status` field on the request remains for fast querying, but the events table is the source of truth for the audit trail.

Lesson: For any entity that has a lifecycle (states it moves through), model the transitions as events from the beginning. The state field is a cache of the latest event. This pattern is the foundation of audit trails, SLA tracking, and analytics.

Discovery: Guest Sessions Are More Powerful Than Guest Accounts

The original design assumed guests would need to create accounts to track their requests. This created friction — guests on a short stay are unwilling to create accounts for a single interaction.

The session model (anonymous session created on QR scan, persisted in localStorage) proved to be both simpler and more powerful. A guest can track all their requests from a single scan without any registration. The session expires with the stay. There is no password to forget and no account to delete.

Lesson: For hospitality use cases, anonymous sessions with a clear expiry are preferable to guest accounts. The QR code is the identity. The session is the continuity.

Frontend Development

Mistake: Building Pages Before Establishing the Layout

In the early phases, individual pages were built before a consistent layout wrapper was established. This led to pages with inconsistent navigation patterns, missing back buttons, and dead-end routes where the user could not return to the previous context.

What we did: Established `DashboardLayout` as the standard wrapper for all admin/staff pages. All new pages are built inside this wrapper.

Lesson: Define the layout and navigation structure before building any pages. A page without an escape route is a trap. The layout is the skeleton; the pages are the organs. Build the skeleton first.

Mistake: Infinite Query Loops from Unstable References

Several pages suffered from infinite re-fetch loops caused by creating new objects or arrays inside the render function and passing them as query inputs. Each render created a new reference, which tRPC interpreted as a changed input, triggering a new fetch.

```
// ❌ This causes infinite re-fetches
const { data } = trpc.rooms.list.useQuery({ ids: [1, 2, 3] });

// ✅ Stabilise with useMemo
const ids = useMemo(() => [1, 2, 3], []);
const { data } = trpc.rooms.list.useQuery({ ids });
```

Lesson: Any object or array passed as a tRPC query input must have a stable reference. Use `useState` for values that are initialised once, and `useMemo` for derived values. This is a React fundamentals issue that is amplified by tRPC's input-change detection.

Discovery: Optimistic Updates Are Essential for List Operations

The initial implementation of list mutations (add item, delete item, toggle status) used `invalidate()` to refresh the list after every mutation. This caused a visible flash — the item would disappear and reappear after the server responded.

Switching to optimistic updates (update the cache immediately, roll back on error) made the UI feel instantaneous. The pattern is more complex to implement but the user experience improvement is significant.

Lesson: For list operations, toggles, and profile edits, always use optimistic updates. For critical operations (payments, auth), use `invalidate` with explicit loading states. The distinction is: optimistic for reversible operations, pessimistic for irreversible ones.

Mistake: Not Accounting for the Sidebar in the Setup Hierarchy Page

When the Setup Hierarchy page's partner card grid was redesigned with a 5-column layout, the grid overflowed the viewport because the sidebar was not accounted for in the column width calculation. The fix required changing to a 3-column layout with responsive breakpoints.

Lesson: Always design responsive grids in the context of the actual layout (sidebar + content area), not in isolation. The available width for a content area inside a dashboard layout is approximately `viewport - sidebar - padding`, not `viewport`. Design for the real container, not the full screen.

Operations & Security

Mistake: Deferring Security Until Late in the Project

OWASP security hardening was done in Phases 42–45, after the core product was built. This meant that security controls (rate limiting, JWT revocation, 2FA, security headers) were retrofitted onto an existing system rather than designed in from the start.

What we did: Added Helmet.js, Redis-backed rate limiting, JWT blacklisting, 2FA, and a full OWASP audit. The system is now compliant, but the retrofit required touching many files.

Lesson: Security is not a phase — it is a constraint. Add security headers, rate limiting, and input validation from the first day. The cost of retrofitting security is always higher than the cost of building it in.

Principle: Redis Is the Right Tool for Ephemeral Security State

JWT revocation, rate limit counters, 2FA tokens, and session blacklists all share a common property: they are ephemeral (they expire), they need to be checked on every request (they must be fast), and they do not need to be durable (losing them on a Redis restart is acceptable). Redis is the correct tool for all of these use cases.

Lesson: Do not store ephemeral security state in the primary database. The primary database is for durable business data. Redis is for fast, expiring security state.

Discovery: The Inbox Is a Product, Not a Feature

The notification inbox started as a simple bell icon with a dropdown. Over 8 phases (Phase 30–76), it evolved into a full email-style inbox with envelope rows, a detail pane, archive management, group collapse, property filtering, mute toggle, sound alerts, and batch dismiss.

This evolution revealed that the inbox is not a notification delivery mechanism — it is the primary interface through which staff manage their workload. Every request that needs attention appears in the inbox. The inbox is the front office, not the front office page.

Lesson: In an interrupt-driven operational system, the notification inbox is the most important surface. Design it as a first-class product from the beginning, not as a utility feature bolted on later.

Copywriting & Communication

Principle: Every Status Must Describe the Next Action

Status labels that only name the current state ("Pending", "In Progress") are less useful than labels that describe what happens next ("Awaiting staff confirmation", "Your request is being prepared"). The guest tracking page was rewritten in Phase 78 to follow this principle throughout.

Lesson: Write status labels from the user's perspective, not the system's perspective. The user does not care what state the system is in — they care what is going to happen next and what they need to do (if anything).

Mistake: Accumulating Stale Copy Over 78 Phases

As the platform evolved, many labels, tooltips, empty states, and error messages became stale — they referred to features that had been renamed, workflows that had been changed, or concepts that no longer existed. The Phase 78 copywriting audit found over 80 issues across 10 files.

Lesson: Copy is code. It needs to be maintained with the same discipline as code. Every time a feature is renamed or a workflow is changed, the copy that describes it must be updated in the same commit. Stale copy is a form of technical debt that erodes user trust.

Summary Table

Category	Mistake / Discovery / Principle	Impact	Phase Fixed
Architecture	Python/FastAPI backend	High — type drift, maintenance burden	Phase 70
Architecture	Hardcoded property IDs	Critical — production data scoping failure	Phase 24
Architecture	Demo data in component state	Medium — false completeness	Phases 25–30
Architecture	No CI from the start	High — regressions not caught	Phase 28
Data	No request event log	High — no audit trail, no SLA tracking	Phase 15+
Frontend	Pages before layout	Medium — navigation dead-ends	Phase 6
Frontend	Unstable query references	High — infinite re-fetch loops	Multiple
Frontend	5-column grid in sidebar layout	Low — visual overflow	Phase 77c
Security	Security deferred to late phases	High — retrofit cost	Phases 42–45
Copy	Stale labels after 78 phases	Medium — user confusion	Phase 78
Discovery	SSE > WebSockets for this pattern	Positive — simpler, more reliable	Phase 19
Discovery	Anonymous sessions > guest accounts	Positive — zero friction for guests	Phase 7
Discovery	Inbox is a first-class product	Positive — core staff workflow surface	Phases 30–76

Document authored by Manus AI in collaboration with Chawakit Sangcharoon (Product Director), March 2026.

Peppr Around — Phase Chronicle

> *A complete timeline of every major feature, pivot, and milestone from the first commit to Phase 78. This is the project's memory.*

How to Read This Document

The chronicle is organised into **Eras** — groups of phases that share a common strategic theme. Within each era, phases are listed with their key deliverables and the strategic context that drove them. Bug fix phases are noted but not detailed unless they represent a significant learning.

Era 1: Foundation (Phases 1–10)

Building the scaffold, establishing the hierarchy, and connecting to real data.

Phase 1–5: Scaffold & Static UI The project began as a static React application with hardcoded demo data. The initial scaffold established the three-level hierarchy (Partner → Property → Room) as the core data model, the DashboardLayout with sidebar navigation, and the basic page structure for the admin back-office. Authentication was mocked. All data was hardcoded arrays.

Phase 6–10: Live Data Integration The first real backend connections were made. The FastAPI (Python) backend was integrated via a REST API client. Pages were progressively wired to real endpoints: QR Management, Front Office, Rooms, and the Setup Hierarchy. The `useActiveProperty()` hook was introduced to replace hardcoded `"pr-001"` property IDs. The onboarding wizard was wired to live room and template counts.

Key milestone: The platform moved from a static mockup to a live-data application. The gap between what the UI showed and what the database contained was closed.

Era 2: Guest Experience (Phases 11–20)

Building the guest-facing microsite and the QR scan flow.

Phase 11–13: Guest Microsite Foundation The guest-facing QR scan flow was built: `ScanLandingPage` (validates the QR code and creates a session), `ServiceMenuPage` (the guest's service catalog with cart), and `TrackRequestPage` (live status tracking). The guest session model (anonymous, localStorage-based, no account required) was established as the identity model for all guest interactions.

Phase 14–16: Guest CMS A mini-CMS was built inside the admin back-office allowing property managers to customise the guest microsite: banner images, welcome messages, property branding, and feature flags. The `peppr_property_config` and `peppr_property_banners` tables were added to the schema.

Phase 17–20: Guest Internationalisation The guest microsite was internationalised to support 8 languages: English, Thai, Chinese, Japanese, Korean, French, German, and Russian. A language selector was added to the guest session. All guest-facing strings were extracted into a translation map.

Key milestone: The guest experience was complete end-to-end: scan → browse → request → track. The platform was demonstrable to hotel operators.

Era 3: Stabilisation & Documentation (Phases 21–28)

Cleaning up technical debt, establishing documentation standards, and setting up CI.

Phase 21–22: Server & Resource Normalization The URL structure was rationalised to use path-based portal separation (`/admin/*`, `/fo/*`, `/sp/*`, `/so/*`, `/guest/*`) instead of subdomains. Legacy API aliases were removed. A route map document was created.

Phase 23–27: Architecture Documentation A formal documentation suite was established: `docs/routes.md` (complete route table), `docs/schema.md` (auto-generated from Drizzle schema), and three Architecture Decision Records (ADRs):

- ADR-001: Why tRPC over REST
- ADR-002: Why MySQL/TiDB over SQLite

- ADR-003: Why SSE over WebSockets

Phase 28: CI Gate The GitHub Actions CI pipeline was set up with two jobs: Unit Tests (no server required) and Integration Tests (live server). The `pnpm test` command was added to the project. The first 233 unit tests were written. The CI gate was set as a required check on the `main` branch.

Key milestone: The project had a formal documentation suite and an automated quality gate. The codebase was no longer held together by manual discipline alone.

Era 4: Front Office Operations (Phases 29–38)

Building the operational heart of the platform — the front office request management system.

Phase 29–31: Front Office Foundation The Front Office portal was built as a distinct operational surface. The request queue, request detail page, and status transition actions (Confirm, Start, Complete, Reject) were implemented. Server-Sent Events were added for real-time queue updates — new requests appear in the queue without a page refresh.

Phase 32–34: Alert Management & Inbox A notification inbox was built as the primary ambient awareness surface for front office staff. Notifications were grouped by property, with a mute toggle and a dismiss action. Quick-action buttons (Confirm, In Progress) were added directly to inbox rows so staff could act without navigating to the detail page.

Phase 35–38: Inbox Polish The inbox was progressively enhanced: dialog deep-links (clicking a notification opens the request detail in a dialog overlay), property filter chips, notification row property subtitles, and a persistent unread count badge on the bell icon.

Key milestone: Front office staff could manage their entire workload from the inbox without navigating to individual request pages. The inbox became the primary operational surface.

Era 5: Security Hardening (Phases 39–48)

A systematic OWASP audit and the implementation of production-grade security controls.

Phase 39–41: GitHub Repository Setup The project was connected to a GitHub repository (`CiaoKit1444/Around-M-v01`). Branch protection rules were configured. The CI pipeline was connected to the repository.

Phase 42–45: OWASP Security Audit A full OWASP Top 10 audit was conducted. Findings were addressed in four phases:

- Phase 42: High severity — SQL injection prevention (Drizzle parameterised queries), XSS prevention (React escaping + CSP)
- Phase 43: Medium severity — Security headers (Helmet.js), CORS configuration
- Phase 44: Medium severity — Body size limits, JWT revocation (Redis blacklist), 2FA foundation
- Phase 45: Final batch — Rate limiting (Redis counters), password policy enforcement, hardcoded domain removal

Phase 46–48: 2FA Implementation Two-factor authentication was implemented end-to-end: TOTP (authenticator app) setup flow, SMS OTP as a fallback, 2FA enforcement per user, backup codes, and an admin UI for managing 2FA status across the user base. A recovery flow was added for users who lose access to their 2FA device.

Key milestone: The platform was OWASP-compliant and ready for production deployment with real guest data.

Era 6: Identity & Branding (Phases 49–62)

Logo rollout, Redis production setup, and the Secret Chamber super-admin bootstrap.

Phase 49–52: Logo Rollout The Peppr Around logo was introduced and rolled out across the platform. Several iterations were required to handle dark background rendering, sidebar sizing, and favicon generation.

Phase 54–55: Redis Production Enhancements Redis was upgraded from a development dependency to a production-grade service with connection pooling, graceful degradation (the platform continues to function if Redis is temporarily unavailable, with reduced security guarantees), and health check endpoints.

Phase 56–58: Logo & QR Simulator Bug Fixes A series of bug fixes addressed: a broken QR scan simulator (used for testing without a physical QR code), a broken sidebar logo (caused by a Vite asset path issue), and a logo rendering failure across all pages (caused by a CDN URL change).

Phase 59–61: Portal Naming & Metric Consistency The portal naming was standardised across the UI. Metric labels (request counts, completion rates, SLA compliance) were made consistent between the dashboard, the front office page, and the analytics reports.

Phase 62: Secret Chamber A hidden super-admin bootstrap route was implemented — a one-time setup page accessible only via a secret URL that allows the first super-admin account to be created without requiring an existing admin to grant the role. This solved the chicken-and-egg problem of bootstrapping a new deployment.

Key milestone: The platform had a complete identity (logo, branding, naming) and a production-grade security infrastructure.

Era 7: Technology Migration (Phases 67–71)

The most significant architectural change in the project's history — migrating from Python/FastAPI to pure TypeScript.

Phase 67: CI Fix & Production Guard A CI failure caused by missing GitHub secrets was diagnosed and fixed. A production guard was added to prevent test data from being written to the production database.

Phase 68–69: Service Template Preview A service template preview card was added to the room detail page, allowing property managers to see exactly what a guest would see before assigning a template to a room. Service count badges and duration display were added to the template list.

Phase 70: Pure JS/TS Stack Cleanup The Python/FastAPI backend was removed. All API endpoints were migrated to tRPC procedures. The `server/routers.ts` and

`server/crudRouter.ts` files became the single source of truth for all backend logic. The `useApi.ts` hook (which wrapped the old REST client) was replaced with direct `trpc.*` calls. The type safety gap between frontend and backend was permanently closed.

Phase 71: Template Preview Card, Service Count Badge & Live Analytics The analytics dashboard was wired to live data. The QR analytics page, revenue report, and satisfaction report were connected to real database queries.

Key milestone: The platform was a pure TypeScript monorepo. The Python backend was gone. Type safety was end-to-end.

Era 8: Inbox Evolution (Phases 72–76)

The inbox grew from a notification panel into a full email-style operational surface.

Phase 72: Inbox & Bell Icon Fixes A series of inbox rendering bugs were fixed: the bell icon badge count was not updating correctly, the inbox panel was not closing when clicking outside, and the unread count was not persisting across page navigations.

Phase 73: Sessions Tab & Cross-Session Persistence A “Sessions” tab was added to the inbox showing active guest sessions at the property. Cross-session persistence was implemented so that inbox state (read/unread, muted properties) survived page refreshes.

Phase 74: Auto-Refresh, Archive & Grouping The inbox was enhanced with: auto-refresh (polling every 30 seconds for new notifications), an Archive tab (notifications older than 24 hours are automatically archived), and group collapse (notifications from the same property can be collapsed into a single row).

Phase 75: Sound Alert, Archive Expiry & Group Dismiss-All A sound alert was added for new high-priority notifications. Archive expiry was implemented (archived notifications are deleted after 7 days). Group dismiss-all was added to the inbox toolbar.

Phase 76: Email-Style Inbox UI The inbox was completely redesigned as an email-style interface. Each notification became an `EnvelopeRow` (avatar, bold subject, preview snippet, timestamp, unread dot). Clicking a row slides into a `DetailPane`

showing the full message body, metadata strip, and action buttons (Confirm / In Progress / View Detail). The inbox became a first-class operational surface, not a notification panel.

Key milestone: The inbox was the most evolved component in the platform. It had grown from a simple bell dropdown into a full email-style inbox with 8 phases of iteration.

Era 9: Setup Hierarchy Usability (Phases 77–78)

Making the hierarchy management interface fast, searchable, and visually clear.

Phase 77: Setup Hierarchy Usability The Setup Hierarchy page (Partner/Property/Room management) was redesigned with: a paginated card grid (3 columns × 2 rows, 6 cards per page), a `HierarchyToolbar` component (debounced search + Sort By selector + Asc/Desc toggle), server-side pagination, and stale-while-revalidate caching with background prefetch. The backend was upgraded to support dynamic `sortBy` (ID / Name / Last Update) for all three entities.

Phase 77d: Elasticsearch-Style Search The search logic was upgraded from exact-match `LIKE '%query%'` to multi-token fuzzy matching: the query is split into tokens, each token is matched across all searchable fields with OR, and all tokens are combined with AND. This matches the Elasticsearch `multi_match` with `operator: AND` pattern.

Phase 77e: Search UX Enhancements Three search UX improvements were added simultaneously: token highlighting (matched words wrapped in amber `<mark>` spans), recent searches dropdown (last 5 queries from localStorage, per entity), and a global Spotlight search (⌘K / Ctrl+K) that queries all three entities simultaneously and groups results by type.

Phase 78: Platform-Wide Copywriting Audit A full audit of all user-facing text across 10 files identified over 80 copy issues. All status labels, action buttons, empty states, error messages, and form placeholders were rewritten to be direct, unambiguous, and action-oriented. The guest tracking page's status descriptions were rewritten to describe what happens next, not just the current state.

Key milestone: The platform was operationally polished. The hierarchy management interface was fast and searchable. The copy was clear and consistent.

Project Statistics at Phase 78

Metric	Value
Total phases completed	78 (+ sub-phases)
Git commits	231
Lines of TypeScript/TSX	84,000
Database tables	32
tRPC procedures	85
Unit tests	233
Pages (admin + guest)	45
Supported guest languages	8
CI pipeline jobs	2 (unit + integration)
OWASP compliance	Full Top 10 addressed

Document authored by Manus AI in collaboration with Chawakit Sangcharoon (Product Director), March 2026.

Peppr Around — Design Language

> *"Design is not decoration. It is the system through which users understand what they can do, what is happening, and what they should do next."*

Philosophy

The Peppr Around design language is built on a single premise: **operational clarity over aesthetic novelty**. The platform is used by hotel staff under time pressure, in noisy environments, on screens of varying quality. Every design decision must pass the question: *“Does this help a staff member act faster and with more confidence?”*

This does not mean the platform is plain. It means that visual richness is earned — it must serve a functional purpose. Colour communicates status. Weight communicates hierarchy. Spacing communicates grouping. Animation communicates state change. Nothing is decorative.

Visual Identity

Colour System

The platform uses a **dark-first** design with a purple accent. The dark background reduces eye strain during long shifts. The purple accent is distinctive and memorable — it is the Peppr brand colour.

Token	Value	Usage
`--background`	`#0F0F11` (near-black)	Page background
`--card`	`#1A1A1F`	Card and panel surfaces
`--border`	`#2A2A35`	Subtle dividers
`--foreground`	`#F0F0F5`	Primary text
`--muted-foreground`	`#8A8A9A`	Secondary text, labels
`--primary`	`#7C3AED` (Peppr Purple)	Primary actions, active states
`--primary-foreground`	`#FFFFFF`	Text on primary backgrounds
`--accent`	`#9D5CF5`	Hover states, highlights
`--destructive`	`#EF4444`	Delete, revoke, reject actions
`--success`	`#22C55E`	Completed, active, confirmed states
`--warning`	`#F59E0B`	Pending, attention required
`--info`	`#3B82F6`	Informational, in-progress

Status colour mapping — this is the most critical part of the colour system. Every status in the request lifecycle has a consistent colour:

Status	Colour	Rationale
Pending	Amber (`--warning`)	Requires attention — warm, urgent
Confirmed	Blue (`--info`)	Acknowledged — calm, in motion
In Progress	Purple (`--primary`)	Active — brand colour, energy
Completed	Green (`--success`)	Done — positive, resolved
Rejected	Red (`--destructive`)	Terminal negative state
Cancelled	Grey (`--muted`)	Terminal neutral state

Typography

The platform uses **Geist** (by Vercel) as the primary typeface. Geist is a geometric sans-serif designed for screen readability at small sizes — essential for dense data tables and card grids.

Scale	Size	Weight	Usage
`text-xs`	12px	400	Timestamps, metadata, badges
`text-sm`	14px	400	Body text, table cells, form labels
`text-base`	16px	400	Default body text
`text-lg`	18px	600	Section headers, card titles
`text-xl`	20px	700	Page titles
`text-2xl`	24px	700	Dashboard metric numbers

Spacing

The platform uses an **8px base grid**. All spacing values are multiples of 8px (or 4px for fine adjustments). This creates visual rhythm and prevents the “random spacing” problem where every component has slightly different padding.

Component Patterns

The Card

Cards are the primary container for entity data (partners, properties, rooms, requests). A well-formed card has:

- A **header** with the entity name (bold, truncated at 1 line) and a status badge
- A **body** with 2–4 key metadata fields (icon + label pairs)
- A **footer** with action buttons (Edit, View, Delete)
- A **selected state** with a solid purple fill and white text (not just a border change)

The selected state deserves special attention. In the Setup Hierarchy, selecting a partner highlights all its properties. The selected card must be visually unmistakable — a border change is not enough. The card background must change.

The PageHeader

Every page has a `PageHeader` component at the top with:

- A **title** (the page name, not a generic label)
- A **subtitle** (one sentence describing what this page is for)
- **Action buttons** on the right (primary action first, secondary actions after)
- An optional **breadcrumb** for pages that are nested within a hierarchy

The subtitle is not optional. It is the first thing a new staff member reads. It must answer the question: *"What do I do on this page?"*

The EmptyState

Every list or grid has an empty state. A well-formed empty state has:

- An **icon** (relevant to the entity type)
- A **title** (what is missing, e.g. "No partners yet")
- A **description** (why it is empty and what to do, e.g. "Add your first partner to start setting up properties and rooms")
- A **call-to-action button** (the primary action that creates the first item)

An empty state without a call-to-action is a dead end. The user must always know what to do next.

The HierarchyToolbar

The `HierarchyToolbar` component is used on all three hierarchy management pages. It has three elements in a single row:

- A **search input** (debounced 300ms, with recent searches dropdown on focus)
- A **Sort By selector** (ID / Name / Last Update)
- An **Asc/Desc toggle button** (arrow icon, toggles direction)

The toolbar is always visible — it does not collapse or hide. Staff should be able to filter and sort without scrolling.

The Inbox EnvelopeRow

The inbox uses an email-style envelope row pattern. Each row has:

- An **avatar** (property initial letter, coloured by property)
- A **subject line** (bold if unread, normal weight if read)
- A **preview snippet** (first 60 characters of the notification body, muted text)
- A **timestamp** (relative time, e.g. “3m ago”)
- An **unread dot** (small purple circle, visible only if unread)

Clicking a row slides in a `DetailPane` from the right. The detail pane has the full message body, a metadata strip (property, room, request number, timestamp), and action buttons.

Layout Principles

The Dashboard Layout

All admin and staff pages use `DashboardLayout` — a persistent sidebar on the left and a scrollable content area on the right. The sidebar contains:

- The Peppr logo at the top
- Navigation groups (Overview, Operations, Service Management, etc.)
- The current user’s name and role at the bottom
- A sign-out button

The content area has a `PageHeader` at the top and the page content below. There is no secondary navigation inside the content area — all navigation is in the sidebar.

The Responsive Grid

Card grids use a responsive column system:

Viewport	Columns	Cards per page
< 600px (mobile)	1	6
600–900px (tablet)	2	6
900px+ (desktop)	3	6

The page size of 6 (3 × 2) is the canonical desktop layout. On mobile, 6 cards in a single column is a long scroll — consider reducing to 4 on mobile in a future iteration.

The Guest Microsite

The guest microsite uses a **mobile-first, single-column layout** with a fixed header (property name + logo) and a fixed footer (cart summary). The content area scrolls. The design is intentionally simpler than the admin back-office — guests are on mobile, in a hurry, and unfamiliar with the platform.

Interaction Principles

Feedback Is Mandatory

Every user action must produce immediate feedback. The three feedback types are:

- **Optimistic update** — the UI changes immediately, before the server responds (for reversible operations)
- **Loading state** — a spinner or skeleton appears while the server is processing (for irreversible operations)
- **Toast notification** — a brief message confirms success or explains failure (for all mutations)

An action without feedback is an action that the user will repeat, assuming it did not work.

Destructive Actions Require Confirmation

Any action that deletes, revokes, or permanently changes data must be confirmed in a dialog. The dialog must:

- Name the specific item being affected (not just “this item”)
- Describe the consequence (not just “are you sure?”)
- Have a clearly destructive primary button (red, labelled with the action verb, e.g. “Revoke QR Code”)
- Have a clearly safe secondary button (“Cancel”)

Progressive Disclosure

Complex workflows are broken into steps. The user sees only what they need to act on at each step. The Setup Hierarchy is the canonical example: the user selects a partner, then sees that partner’s properties, then selects a property, then sees that property’s rooms. The full hierarchy is always visible in the breadcrumb, but the user acts on one level at a time.

Copy Principles

Verb-First Action Labels

All action buttons use verb-first imperative phrasing. The verb names the action, not the object.

Avoid	Use instead
“QR Code” (button)	“Generate QR Code”
“Partner” (button)	“Add Partner”
“Submit” (generic)	“Submit Service Request”
“OK”	“Confirm” or the specific action

Status Labels Describe What Happens Next

Status labels must answer: *"What is happening right now, and what happens next?"*

Avoid	Use instead
"Pending"	"Awaiting staff confirmation"
"In Progress"	"Your request is being prepared"
"Completed"	"Your request has been fulfilled"

Empty States Are Invitations

Empty state messages must invite the user to take action, not just report the absence of data.

Avoid	Use instead
"No partners found"	"No partners yet — add your first partner to start setting up properties and rooms"
"No requests"	"All clear — no pending requests at this property"

Accessibility Baseline

The platform targets **WCAG 2.1 AA** compliance. The minimum requirements are:

- Colour contrast ratio ≥ 4.5:1 for body text, ≥ 3:1 for large text
- All interactive elements reachable by keyboard (Tab, Enter, Space, Escape)
- Visible focus rings on all focusable elements
- All images have `alt` text
- Form inputs have associated `label` elements

The dark theme with Peppr Purple accent was tested for contrast compliance. The amber warning colour (`#F59E0B`) on a dark background passes AA at body text sizes.

Document authored by Manus AI in collaboration with Chawakit Sangcharoon (Product Director), March 2026.

Peppr Around — Rebirth Playbook

> *"If we had to build this again from scratch, knowing everything we know now, this is exactly how we would do it."*

Purpose

This playbook is the operational guide for rebuilding the Peppr Around platform from zero. It is written for a team that has read the Manifesto, Architecture, Lessons Learned, and Design Language documents and is ready to execute. It assumes a two-person team (one product/design, one engineering) with access to the Manus AI development environment.

The playbook is structured as a sequence of **Sprints**, each with a clear goal, deliverables, and a definition of done. The order is deliberate — each sprint builds on the previous one and avoids the mistakes documented in the Lessons Learned file.

Pre-Sprint: Environment Setup

Before writing a single line of application code, the following must be in place.

Technology stack (non-negotiable):

- Runtime: Node.js 22 LTS
- Language: TypeScript 5.x (strict mode)
- Frontend: React 19 + Vite + Tailwind CSS 4
- Backend: Express 4 + tRPC 11
- Database: MySQL/TiDB (Drizzle ORM)

- Cache/Security state: Redis
- Testing: Vitest
- CI: GitHub Actions

Repository setup checklist:

1. Create the GitHub repository with branch protection on `main` (require CI to pass before merge)
2. Set up the GitHub Actions workflow immediately — do not defer this (see `02-LESSONS-LEARNED.md`)
3. Add all required secrets to GitHub Actions: `JWT_SECRET`, `SSO_BRIDGE_SECRET`, `DATABASE_URL`, `REDIS_URL`
4. Create `.env.example` listing every required environment variable with a description but no value
5. Set up the Manus project with `web-db-user` capabilities

First commit must include:

- The tRPC scaffold (routers.ts, crudRouter.ts, context.ts)
- The Drizzle schema with the three-level hierarchy tables
- The DashboardLayout component
- The CI workflow file
- The `.env.example` file
- A passing `pnpm test` with at least one smoke test

Sprint 1: Data Model & Authentication (Week 1)

Goal: The database schema is complete, migrations run cleanly, and authentication works end-to-end.

Why this first: The data model is the foundation of everything. Every feature built without a stable schema will require migration work later. Authentication must be in place before any protected routes are built.

Deliverables:

The complete Drizzle schema with all 32 tables, pushed to the database via `pnpm db:push`. The schema must include:

- `partners` — the top level of the hierarchy
- `properties` — scoped to a partner
- `rooms` / `service_units` — scoped to a property
- `service_templates` and `service_template_items` — the service catalog
- `peppr_service_requests` — the request lifecycle table
- `peppr_request_events` — the immutable event log (see `02-LESSONS-LEARNED.md`)
- `peppr_guest_sessions` — anonymous guest sessions
- `users` with `role` (admin/user/super_admin)
- `peppr_property_config` — per-property CMS settings
- `qr_codes` — QR binding table

The authentication flow must be complete: Manus OAuth login, JWT session cookie, `protectedProcedure` middleware, and the `useAuth()` hook on the frontend.

Definition of done: A logged-in admin can see their name in the sidebar. `pnpm check` reports 0 errors. `pnpm test` passes.

Sprint 2: Setup Hierarchy (Week 2)

Goal: An admin can create and manage Partners, Properties, and Rooms through the UI.

Why this second: The hierarchy is the master data. Every other feature (requests, QR codes, service templates) depends on having partners, properties, and rooms in the database. Build the hierarchy management before building anything that uses it.

Deliverables:

The Setup Hierarchy page (`/admin/onboarding`) with three sections:

- Partners section: card grid (3 columns, 6/page), HierarchyToolbar (search + sort), Add/Edit/Delete partner
- Properties section: card grid filtered by selected partner, Add/Edit/Delete property
- Rooms section: table view filtered by selected property, Add/Edit/Delete room, bulk seed

The `HierarchyToolbar` component must be built as a reusable component from the start — do not inline the search/sort logic into each page.

The backend must support:

- `crud.partners.list` with `search`, `sortBy`, `sortOrder`, `page`, `pageSize`
- `crud.properties.list` with the same params plus `partner_id` filter
- `crud.rooms.list` with the same params plus `property_id` filter
- Multi-token fuzzy search (see `02-LESSONS-LEARNED.md` — build this correctly from the start)

Definition of done: An admin can create a partner, add a property to it, and add 10 rooms to the property. The search finds rooms by partial name match across multiple tokens.

Sprint 3: Service Templates (Week 3)

Goal: An admin can create service templates and assign them to rooms.

Deliverables:

The Service Templates page (`/admin/service-templates`) with:

- Template list with search and sort
- Template detail with item management (add/edit/delete items, set price, duration, image)
- Template assignment to rooms (bulk assign from the room table)

The backend must support:

- `crud.serviceTemplates.list`, `create`, `update`, `delete`
- `crud.serviceTemplateItems.list`, `create`, `update`, `delete`
- `crud.rooms.assignTemplate` (bulk assign)

Definition of done: An admin can create a template with 5 items, assign it to 10 rooms, and see the template name on each room card.

Sprint 4: QR Code Management (Week 4)

Goal: QR codes can be generated, bound to rooms, and printed.

Deliverables:

The QR Management page (`/admin/qr`) with:

- QR code list with search and filter by property/status
- Bulk generate QR codes for unbound rooms
- Individual QR code detail with download (PNG, PDF) and print
- QR binding status (bound/unbound) on each room card in the hierarchy

The backend must support:

- `crud.qrCodes.list`, `generate`, `bind`, `unbind`, `revoke`
- QR code validation endpoint (used by the guest microsite)

Definition of done: An admin can generate QR codes for all rooms in a property, download a PDF with all codes, and scan one with a phone to reach the guest landing page.

Sprint 5: Guest Microsite (Week 5)

Goal: A guest can scan a QR code, browse the service menu, submit a request, and track it.

Deliverables:

Four guest pages:

- `ScanLandingPage` — validates the QR token, creates a guest session, redirects to the service menu
- `ServiceMenuPage` — the service catalog with categories, items, cart, and checkout
- `TrackRequestPage` — live request status with SSE updates
- `ServiceMenuPage` (order history) — previous requests for the current session

The guest session model must be anonymous (no account required). The session is created on QR scan and stored in localStorage. The session ID is passed with every guest API call.

The backend must support:

- `guest.validateQr` — validates the QR token, creates a session
- `guest.getMenu` — returns the service template for the room
- `guest.submitRequest` — creates a service request
- `guest.trackRequest` — returns the current status of a request (with SSE subscription)

Definition of done: A guest can scan a QR code, order a room service item, and see the status update from “Awaiting confirmation” to “Your request is being prepared” in real time.

Sprint 6: Front Office Operations (Week 6)

Goal: Front office staff can manage the request queue, confirm requests, and track completion.

Deliverables:

The Front Office portal (`/fo`) with:

- Request queue with real-time updates (SSE)

- Request detail page with status transition actions (Confirm, Start, Complete, Reject)
- Request notes (staff can add internal notes to any request)
- SLA deadline display (amber when < 15 minutes, red when overdue)

The notification inbox with:

- Envelope-row list (subject, preview, timestamp, unread dot)
- Detail pane (full message, metadata, action buttons)
- Archive tab
- Property filter
- Mute toggle

The backend must support:

- `requests.list` with status filter and property filter
- `requests.updateStatus` with actor, timestamp, and reason (appended to `peppr_request_events`)
- `requests.addNote`
- SSE endpoint for real-time queue updates

Definition of done: A front office staff member can see a new request appear in the queue within 3 seconds of submission, confirm it, and see the guest's tracking page update in real time.

Sprint 7: Security Hardening (Week 7)

Goal: The platform is OWASP Top 10 compliant and ready for production.

Do not defer this sprint. Security hardening done after the product is built is always more expensive than security built in from the start. This sprint must happen before any real guest data is processed.

Deliverables:

- Security headers via Helmet.js (CSP, HSTS, X-Frame-Options, etc.)

- Rate limiting via Redis (per-IP for public endpoints, per-user for authenticated endpoints)
- JWT revocation via Redis blacklist (logout invalidates the token immediately)
- 2FA via TOTP (authenticator app) with SMS OTP fallback
- Input validation on all tRPC procedures (Zod schemas)
- Body size limits on all upload endpoints
- CORS configuration (explicit allowlist, not wildcard)
- SQL injection prevention (Drizzle parameterised queries — already handled by the ORM, but verify)

Definition of done: An OWASP ZAP scan reports no high or critical findings. The `pnpm test` suite includes tests for rate limiting and JWT revocation.

Sprint 8: Analytics & Reporting (Week 8)

Goal: Property managers can see operational metrics and export reports.

Deliverables:

The Analytics dashboard with:

- Request volume over time (daily/weekly/monthly)
- Average response time and SLA compliance rate
- Top requested services
- Revenue by service category (if pricing is enabled)
- Guest satisfaction scores (if feedback is enabled)

The Reports page with:

- Exportable CSV/PDF reports for each metric
- Date range selector
- Property filter

Definition of done: A property manager can see the request volume for the past 30 days and export a CSV report.

Sprint 9: Guest CMS & Internationalisation (Week 9)

Goal: Property managers can customise the guest microsite and guests can use it in their preferred language.

Deliverables:

The Guest CMS in the admin back-office:

- Property branding (logo, banner image, welcome message)
- Feature flags (enable/disable service categories)
- Language settings (default language for the property)

Guest microsite internationalisation:

- 8 languages: English, Thai, Chinese, Japanese, Korean, French, German, Russian
- Language selector on the guest landing page
- All guest-facing strings extracted into a translation map

Definition of done: A property manager can upload a banner image and set a Thai welcome message. A Thai-speaking guest can switch to Thai and see all UI text in Thai.

Sprint 10: Polish & Launch Readiness (Week 10)

Goal: The platform is production-ready — no stale copy, no broken flows, no missing empty states.

Deliverables:

A full copywriting audit (see `02-LESSONS-LEARNED.md` — do this as a dedicated sprint, not as an afterthought):

- Every status label describes what happens next
- Every empty state has a call-to-action
- Every action button uses verb-first imperative phrasing

- Every error message includes a recovery action

A full accessibility audit:

- Colour contrast ≥ 4.5:1 for all body text
- All interactive elements keyboard-reachable
- All images have `alt` text

A full CI audit:

- All 233+ unit tests passing
- Integration tests passing
- TypeScript: 0 errors
- No hardcoded IDs, URLs, or credentials

Definition of done: The CI pipeline is green. The copywriting audit finds 0 issues. The platform is ready to onboard the first real hotel property.

The Rules That Must Never Be Broken

These are the non-negotiable constraints distilled from 78 phases of experience. They are not guidelines — they are rules.

Rule 1: Never hardcode an entity ID. Not even in a prototype. Use a constant with a clear name that can be found and replaced with a single grep.

Rule 2: Never store demo data in component state. Build the real data layer first. A loading skeleton is more honest than fake data.

Rule 3: Never defer security. OWASP hardening is Sprint 7, not Sprint 20. The cost of retrofitting security is always higher than the cost of building it in.

Rule 4: Never build a page without an escape route. Every page must have a way back. The layout is the skeleton — build it before the pages.

Rule 5: Never use a single `status` field for a lifecycle entity. Add an event log table from the beginning. The status field is a cache of the latest event.

Rule 6: Never write copy as an afterthought. Copy is code. It must be maintained with the same discipline. Every feature rename requires a copy update in the same commit.

Rule 7: Never skip CI setup. The CI pipeline is the first thing set up after the scaffold. Tests written after the fact are always harder to write and less complete.

Rule 8: Never use WebSockets when SSE is sufficient. If the real-time channel is asymmetric (server pushes, client mutates via HTTP), use SSE. It is simpler, more reliable, and reconnects automatically.

Estimated Timeline

Sprint	Focus	Duration
Pre-Sprint	Environment setup	1 day
Sprint 1	Data model + auth	5 days
Sprint 2	Setup hierarchy	5 days
Sprint 3	Service templates	4 days
Sprint 4	QR management	4 days
Sprint 5	Guest microsite	5 days
Sprint 6	Front office ops	6 days
Sprint 7	Security hardening	4 days
Sprint 8	Analytics	4 days
Sprint 9	CMS + i18n	4 days
Sprint 10	Polish + launch	3 days
Total		**~45 working days**

With Manus AI as the primary engineering partner, the actual velocity will be significantly higher than a traditional team. The original platform was built in

approximately 78 phases over several weeks. A rebuild with this playbook and the lessons learned should reach the same feature parity in 3–4 weeks.

Document authored by Manus AI in collaboration with Chawakit Sangcharoon (Product Director), March 2026.

Peppr Around — Future Roadmap

> *"These are the features we knew we wanted, the ideas we validated, and the improvements we discovered — but did not build in v1. This is where v2 begins."*

How to Read This Document

Every item in this roadmap was either explicitly requested, identified during a usability review, or discovered as a natural extension of an existing feature. Items are grouped by domain and prioritised within each group using a **P1 / P2 / P3** scale:

- **P1 — High impact, low complexity.** Build these in the first sprint of v2. They are quick wins that significantly improve the experience.
- **P2 — High impact, medium complexity.** Build these in the second or third sprint. They require design and backend work but are well-understood.
- **P3 — Strategic.** These are larger investments that require product validation before committing engineering time.

Domain 1: Setup Hierarchy

P1 — URL-Persisted Filter State

What: Encode the current search query, page number, sort field, and sort direction into the URL as query parameters (e.g. `?search=grand&page=2&sort=name&order=asc`).

Why: Staff frequently need to return to a specific filtered view after navigating away — for example, after editing a partner and returning to the list. Without URL persistence, the filter state is lost and the user must re-enter it. This is a high-friction pattern that occurs dozens of times per shift.

How: Use `wouter`'s `useSearch()` hook to read and write query params. Debounce writes to avoid polluting the browser history on every keystroke. The `HierarchyToolbar` component already has all the state — it just needs to be synchronised with the URL.

P1 — Active Filter Chips

What: Show dismissible chips below the `HierarchyToolbar` when a non-default filter is active (e.g. `Search: grand`, `Sort: Name ↑`). Each chip has an × button that clears that specific filter.

Why: Staff cannot always tell at a glance whether the list they are looking at is filtered. A chip row makes the active filters explicit and provides a one-click way to clear them without scrolling back to the toolbar.

How: Read the current filter state from the `HierarchyToolbar` props and render a chip row below the toolbar when any filter is non-default. The chip row is zero-height when no filters are active.

P2 — Inline Quick-Edit Popover

What: Add a small popover on the Edit icon of each card that lets staff update the most common fields without navigating to the full edit page. For partners: status, contact person. For properties: city, country. For rooms: template assignment, status.

Why: The most common edit operation is a single-field change (e.g. changing a room's template, updating a partner's status). Navigating to the full edit page for a single-field change is a high-friction pattern. The popover reduces this to two clicks.

How: Use a `Popover` from `@radix-ui/react-popover` triggered by the Edit icon. The popover contains a minimal form with only the most commonly edited fields. On save, it calls the existing `crud.*.update` tRPC mutation and closes.

P2 — Responsive Grid Breakpoints for Service Areas

What: Apply the same `xs/sm/md` responsive column breakpoints to the Service Areas (Properties) section within the Setup Hierarchy page. Currently it uses `auto-fill minmax(220px, 1fr)` which produces inconsistent column counts across screen sizes.

Why: The partner grid and the service areas grid are visually inconsistent on the same page. The partner grid uses the 3-column responsive system; the service areas grid uses a different layout algorithm.

Domain 2: Search & Discovery

P1 — Search Result Highlighting in the Spotlight

What: The global Spotlight (⌘K) already highlights tokens in the section-level search results. Extend the same `HighlightText` component to the Spotlight result rows so that matched tokens are highlighted in the name and subtitle of each result.

Why: When a search matches a secondary field (e.g. a partner's email rather than its name), the user cannot tell why the result appeared. Highlighting the matched text in the result row makes the relevance immediately visible.

P1 — Global Search Result Count Badges

What: Show a small count chip next to each group header in the Spotlight (e.g. "Partners 3 / 4") indicating how many results were returned and how many total exist. If the total exceeds the displayed count, add a "See all →" link that navigates to the full page with the search pre-filled.

Why: Staff need to know whether the Spotlight is showing all matching results or just a sample. Without a count, a user who sees 4 partner results does not know if there are 4 or 40.

P2 — Recent Searches Across Entities

What: Extend the recent searches feature from per-entity (each hierarchy page has its own history) to cross-entity (the Spotlight maintains a unified recent searches history that includes results from all entity types).

Why: Staff often search for the same term across multiple entity types (e.g. “Grand” to find all Grand-branded partners, properties, and rooms). A unified recent searches history in the Spotlight reduces repetitive typing.

P2 — Keyboard Shortcut `/` to Focus Section Search

What: When focus is inside the Setup Hierarchy page, pressing `/` should jump the cursor directly into the currently active section’s search input (partner search if the partner section is active, property search if a partner is selected, etc.).

Why: Power users — particularly property managers who manage large portfolios — navigate the hierarchy dozens of times per shift. The `/` shortcut is a widely recognised convention (GitHub, Linear, Notion) that eliminates the need to reach for the mouse.

Domain 3: Front Office Operations

P1 — Browser Push Notifications

What: When the browser tab is backgrounded or minimised, fall back to the Web Push API to deliver a system-level notification for new high-priority requests. The notification includes the property name, room number, and request type. Clicking it brings the tab to the foreground and opens the request detail.

Why: Front office staff frequently have the dashboard open in a background tab while working in another application. The in-app sound alert and inbox badge are not visible when the tab is backgrounded. Browser push notifications ensure that no high-priority request is missed.

How: Use the `Notification` Web API with `ServiceWorker` registration. Request permission on first login. Store the push subscription in the database against the user record. The server sends a push when a new request is created with `priority: high`.

P1 — Notification Labels / Priority Tags

What: Let staff tag any inbox notification with a coloured label (e.g. Urgent, Follow-up, Done, VIP) directly from the detail pane. Add a label filter chip row above the inbox list so staff can filter to only labelled notifications.

Why: In a busy property with 50+ active requests, staff need a way to mark notifications for follow-up without dismissing them. Labels provide a lightweight triage system without requiring a full task management feature.

P2 — SLA Escalation Alerts

What: When a request's SLA deadline is within 10 minutes and it has not been confirmed, automatically escalate it: change the inbox row background to amber, play a distinct sound alert, and send a push notification to all front office staff at the property.

Why: SLA breaches are the most operationally damaging outcome for a hotel's service reputation. Passive indicators (a red badge) are not sufficient when a shift is busy. Active escalation ensures that no SLA breach goes unnoticed.

P2 — Shift Handoff Summary Email

What: At the end of each shift (configurable time per property), automatically generate a shift summary email and send it to the outgoing shift manager. The email includes: total requests handled, average response time, SLA compliance rate, open requests being handed over, and any notes added during the shift.

Why: The shift handoff page already exists in the UI. Automating the summary email removes the manual step of exporting and sending the report, and ensures continuity between shifts even when the handoff is rushed.

Domain 4: Guest Experience

P1 — Guest Feedback After Completion

What: After a request is marked Completed, the guest receives a push notification (or sees a prompt on the tracking page) asking for a 1–5 star rating and an optional comment. The rating is stored against the request and surfaced in the analytics dashboard.

Why: Guest satisfaction data is the most valuable operational metric for a hotel. Currently the platform has no feedback loop — staff complete requests but never know if the guest was satisfied. A lightweight post-completion rating takes less than 10 seconds for the guest and provides actionable data for property managers.

P1 — Guest Request Amendment

What: Allow guests to amend a submitted request (change quantity, add a note, change preferred time) up until the point it is confirmed by staff. After confirmation, amendments are locked and the guest must contact the front office.

Why: Guests frequently realise they made a mistake immediately after submitting a request. Currently they have no way to correct it without calling the front office. Self-service amendment reduces front office interruptions and improves guest satisfaction.

P2 — Repeat Order (Order Again)

What: On the guest tracking page and order history, add an “Order Again” button that pre-fills the cart with the same items from a previous request. The guest can adjust quantities before submitting.

Why: Repeat orders are common in hospitality (same breakfast order every morning, same towel request). Pre-filling the cart from a previous order reduces the number of taps required from 8–10 to 2–3.

P3 — Native Mobile App (PWA)

What: Package the guest microsite as a Progressive Web App (PWA) with an `Add to Home Screen` prompt. The PWA provides a native-app-like experience: full-screen mode, offline capability for the service menu (cached on first load), and push notifications without requiring a browser tab.

Why: A PWA eliminates the app store barrier while providing a significantly better experience than a browser tab. For guests who stay more than one night, the home screen icon is a persistent reminder of the service.

Domain 5: Administration & Security

P2 — Admin Audit Log UI

What: Build a dedicated Audit Log page in the admin back-office that surfaces the `peppr_request_events` table in a searchable, filterable timeline. Staff can filter by actor, event type, property, and date range. Each event shows the actor, timestamp, previous state, new state, and reason.

Why: The audit log data already exists in the database (added in Phase 15). It is currently only accessible via direct database queries. A UI surface makes it accessible to property managers for compliance and dispute resolution without requiring database access.

P2 — Role-Based Access Control Expansion

What: Expand the current two-role system (admin / user) to a four-role system: Super Admin (full platform access), Property Manager (full access to their assigned properties), Front Office Staff (request management only), and Read-Only (analytics and reports only).

Why: The current system grants all authenticated users the same level of access within their property. In practice, front office staff should not be able to modify service templates or QR codes. Property managers should not be able to modify other properties' settings. The role expansion enforces the principle of least privilege.

P3 — Multi-Tenant White-Label

What: Allow the platform to be deployed as a white-label product for hotel management companies. Each hotel management company gets its own subdomain, logo, colour scheme, and isolated data partition. The super-admin console allows creating and managing tenants.

Why: The three-level hierarchy (Partner → Property → Room) already maps naturally to a multi-tenant model: a hotel management company is a tenant, its brands are partners, its hotels are properties, and its rooms are rooms. The data isolation is already enforced by `property_id` foreign keys. The white-label layer is primarily a branding and routing concern.

Domain 6: Analytics & Reporting

P2 — Exportable Reports (CSV / PDF)

What: Add export buttons to the analytics dashboard that generate a CSV or PDF report for the currently displayed date range and property filter. The PDF report uses the property's branding (logo, colours).

Why: Property managers are required to submit operational reports to hotel owners and management companies. Currently they must screenshot the dashboard or manually copy data. A one-click export reduces this to a single action.

P2 — Cohort Analysis: Guest Return Rate

What: Track whether guests who used the service request system return for a second stay (identified by phone number or email). Show a "Guest Return Rate" metric on the analytics dashboard.

Why: The ultimate measure of the platform's value to a hotel is whether it improves guest retention. A guest who had a seamless service experience is more likely to return. This metric connects the operational data to the business outcome.

P3 — Predictive Demand Forecasting

What: Use historical request data to predict demand for each service category by day of week, time of day, and season. Surface predictions as a “Demand Forecast” widget on the front office dashboard so staff can prepare for busy periods.

Why: Hotels with predictable demand patterns (business hotels with Monday–Thursday peaks, resort hotels with weekend peaks) can significantly improve service quality by staffing proactively rather than reactively.

Prioritisation Summary

Priority	Count	Estimated effort
P1 (Quick wins)	8 items	1–2 days each
P2 (Medium investment)	10 items	3–5 days each
P3 (Strategic)	4 items	2–4 weeks each

Recommended v2 Sprint 1 (P1 items only):

1. URL-persisted filter state in HierarchyToolbar
2. Active filter chips below the toolbar
3. Search result highlighting in the Spotlight
4. Global search result count badges
5. Browser push notifications for high-priority requests
6. Notification labels / priority tags
7. Guest feedback after completion
8. Guest request amendment

These 8 items can be delivered in approximately 10 working days and will produce a measurable improvement in staff efficiency and guest satisfaction.

Document authored by Manus AI in collaboration with Chawakit Sangcharoon (Product Director), March 2026.